Time-Series Data and Predicting Cryptocurrency Prices using Deep Learning

Jesse Gempel
Introduction to Deep Learning

May 7, 2024

Abstract

"Cryptocurrency" is defined by the Oxford Learner's Dictionaries as "any system of electronic money, used for buying and selling online and without the need for a central bank."[1]. It is a recently popularized alternative to stocks and other centralized—or government controlled—forms of investments. Perhaps the most popular example of a cryptocurrency is Bitcoin. Bitcoin gathered significant media attention to its extremely high growth—from less than 1 cent in 2010 to around $16,000 in 2017[2]. Because of Bitcoin's extreme price growth, individuals consistently research price prediction articles online, many of which seem to offer conflicting future price values.

The objective of this project is to predict closing prices for Bitcoin one week into the future. Day-to-day time-series data for Bitcoin will be analyzed by a Deep Learning model to satisfy this objective. It is known that Bitcoin and other cryptocurrencies are extremely volatile compared to traditional stocks. Because crypto prices wildly fluctuate within a short time period, it can be difficult to accurately construct a pricing forecast. This scenario makes for a risky and interesting Deep Learning problem to solve.

An overfitting model was first created to memorize the 5-year closing price history from Bitcoin. A second model was then constructed to offer price predictions while working with unseen data. Afterwards, the model was improved by applying data augmentation and regularization techniques. The final step of this Deep Learning problem is to create and test alternative model architectures. Each one of these strategies' results will be analyzed in this report.

1 Collecting and Normalizing Data

To collect the time-series dataset for this project, I utilized a website called *Yahoo! Finance*[3]. This website contains a .csv dataset called *BTC_USD.csv* which includes day-to-day pricing for Bitcoin within the previous 5 years.. The .csv file was then placed inside of a single folder—that way, it is possible to complete the initial phase of constructing an overfitting model.

The complete data for Bitcoin's day-to-day Adjusted Closing Price is illustrated in Figure 1. The figure was intended to help visualize Bitcoin's unique patternicity. Figure 2 provides more detailed data for the dataset that spans over the previous week (between 4/27/2024 and 5/3/2024). The purpose of these illustrations is to verify that the dataset will properly work with any model for this project.

The process of normalization is important because some data features (or columns in Figure 2) involve a radically different scaling or minimum-maximum range. Normalization was applied by utilizing a MinMaxScalar[4] for each feature. This process allows a model to work with similarly-scaled data, with facilitates the crypto price prediction process.

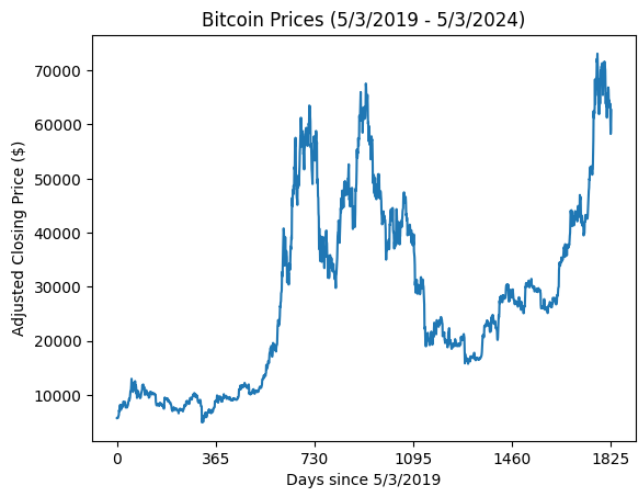

Figure 1: BTC Adjusted Closing Prices between 5/3/2019 and 5/3/2024.

	Date	Open	High	Low	Close	Adj Close	Volume
1822	2024-04-27	63750.988281	63898.363281	62424.718750	63419.140625	63419.140625	19530783039
1823	2024-04-28	63423.515625	64321.484375	62793.597656	63113.230469	63113.230469	17334827993
1824	2024-04-29	63106.363281	64174.878906	61795.457031	63841.121094	63841.121094	26635912073
1825	2024-04-30	63839.417969	64703.332031	59120.066406	60636.855469	60636.855469	37840840057
1826	2024-05-01	60609.496094	60780.500000	56555.292969	58254.011719	58254.011719	48439780271
1827	2024-05-02	58253.703125	59602.296875	56937.203125	59123.433594	59123.433594	32711813559
1828	2024-05-03	59087.492188	62671.398438	58857.058594	62671.398438	62671.398438	31852531712

Figure 2: Tabular data for several Bitcoin metrics. The BTC_USD.csv dataset from Yahoo! Finance is current as of 5/3/2024. In this experiment, we will focus on one particular feature: the Adjusted Closing Price (marked as Adj. Close in the table)

```
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_raw_data = scaler.fit_transform(raw_data)

target_scaler = MinMaxScaler(feature_range=(0, 1))
scaled_targets = target_scaler.fit_transform(crypto_price.reshape(-1, 1)).flatten()
```

Figure 3: Declaration of the MinMaxScalar used for normalizing the data.

2 Overfitting the Dataset

Creating an overfitting time-series model was possible after rigorously adjusting its hyperparameters and layer organization. The overfitting model comprised of 3 LSTM layers: the first layer contained 6 neurons, the second layer had 4, and the third layer had 2. This configuration resulted in a model with only 547 parameters, as shown in Figure 4.

```
Model: "sequential_5"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 lstm_14 (LSTM)              (None, 7, 6)              312

 lstm_15 (LSTM)              (None, 7, 4)              176

 lstm_16 (LSTM)              (None, 2)                 56

 dense_5 (Dense)             (None, 1)                 3

=================================================================
Total params: 547 (2.14 KB)
Trainable params: 547 (2.14 KB)
Non-trainable params: 0 (0.00 Byte)
```

Figure 4: Architecture of the smallest possible overfitting model.

While overfitting the *BTC_USD.csv* dataset, we ensured that the entire set was processed only for training (i.e., no validation or testing). Once the models finished running 100 epochs, the loss and Mean Squared Error (MSE) values approached 1.0 * 10^{-4}, while the Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) values approached 1.0 * 10^{-2}.

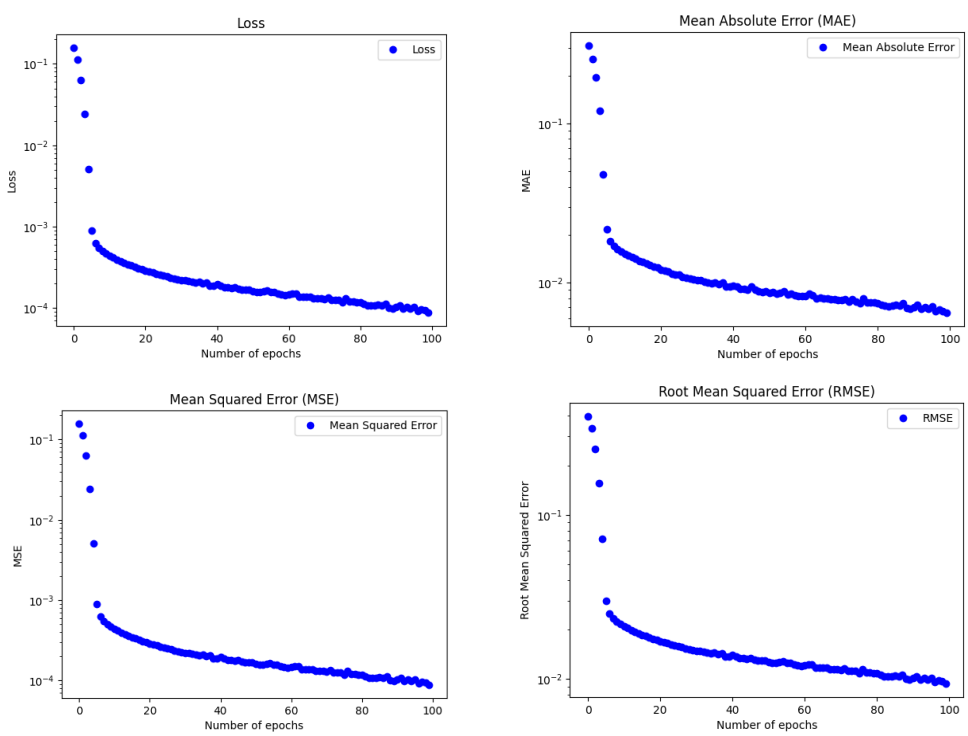

Figure 5: Loss, MAE, MSE, and RMSE measures once the overfitting model completed 100 epochs.

3 Separating the Data into Training, Validation, and Test Sets

Separation of data constituted the next phase of creating a Bitcoin cryptocurrency forecaster. The *BTC_USD.csv* dataset was separated into three additional .csv sets for training, validation, and testing. Separating the dataset was crucial towards ensuring that the model's performance could be thoroughly examined.

The dataset contained one row of Bitcoin price data for each day. The time span of the dataset ranged between 5/3/2019 and 5/3/2024. As a result, there were a total of 1,828 data rows spanning over the previous 5 years of Bitcoin's operation. Division of the data occurred in this manner: 60% was placed in a training .csv file, 20% was located in a validation .csv file, and the remaining 20% was placed in a test .csv file. Data among the three .csv files was mutually exclusive—none of the same data rows appeared in more than one of the sets. Also, it was made sure that the training set contained less recent data, whereas the validation and test sets included more recent data. This information is better summarized in Figure 6.

Dataset	Time Span	Number of Samples	Percent of Data Used
Training	5/3/19 - 5/2/22	1096	60%
Validation	5/3/22 - 5/2/23	365	20%
Test	5/3/23 - 5/3/24	367	20%
Entire Dataset	5/3/19 – 5/3/24	1828	100%

Figure 6: Illustration of the BTC_USD.csv dataset's separation into training, validation, and test sets.

4 Creating a Robust Model

4.1 Data Instantiation and Shuffling

Instantiation of the training, validation, and test sets[5] serves as the basis towards preparing the data for training. Throughout this process, the batch size was reduced from 256 to 64 samples, plus data shuffling was included for each dataset. These two decisions seemed to help the model yield better results.

4.2 Common Sense Baseline

The method of calculating the common sense baseline was based on code from the *Deep Learning with Python* textbook in Listing 10.9[5]. The textbook determined a common sense baseline for an existing model involving the Mean Absolute Error (MAE) However, the code was modified to calculate Mean Squared Error (MSE) and Root Mean Squared Error (RMSE) baselines as well. The baselines are illustrated in Figure 7.

	Validation	Test
MAE	0.06	0.06
MSE	0.01	0.01
RMSE	0.09	0.09

Figure 7: Illustration of common sense baselines for the validation and test sets. These scores will be used to compare the actual loss scores of the model

4.3 Callbacks

Two callback methods were added to the model: ModelCheckpoint and EarlyStopping. ModelCheckpoint saved the best possible weights from epochs with a low validation loss score. EarlyStopping prematurely ceased the model from running if the validation loss score did not improve after 20 epochs. These two callback methods serve as guards against overfitting.

4.4 The Ideal Model

After experimenting with layer count, neuron count, and loss activation type, the best performing model was one with 3 LSTM layers and 15,651 parameters (as shown in Figure 8). Determination of the best performing model was made under the educated guess that LSTMs (Long Short-Term Memory layers) would serve as the best model type for this experiment, though no other model types were examined at this point.

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 lstm (LSTM)                 (None, 7, 40)             7520

 lstm_1 (LSTM)               (None, 7, 20)             4880

 lstm_2 (LSTM)               (None, 10)                1240

 dense (Dense)               (None, 1)                 11

=================================================================
Total params: 13651 (53.32 KB)
Trainable params: 13651 (53.32 KB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

Figure 8: Architecture of the best performing model once the data was split.

The validation loss for the best model was 0.0076, its validation MAE was 0.0603, its validation MSE was 0.0076, and its validation RMSE was 0.0890. Even after adjusting the model's hyperparameters and architecture, the latter three metrics do not fare well against the common sense baselines, as shown in Figure 9.

	Ideal Model's Performance	**Common Sense Baseline**
MAE	0.0603	0.06
MSE	0.0076	0.01
RMSE	0.0890	0.09

Figure 9: Comparison of the model's metrics to its common sense baselines. For each of these scoring metrics, a lower number indicates better performance. Red indicates that the model could not beat the baseline. Yellow indicates that the model *barely* beats the baseline. Green demonstrates a considerable performance against the baseline. These results signify a low confidence that predicting Bitcoin prices that Deep Learning can effectively offer a cryptocurrency forecast due to its volatile nature. However, it is important to improve the model, either through augmentation, regularization, or other model types, to be sure of this conclusion.

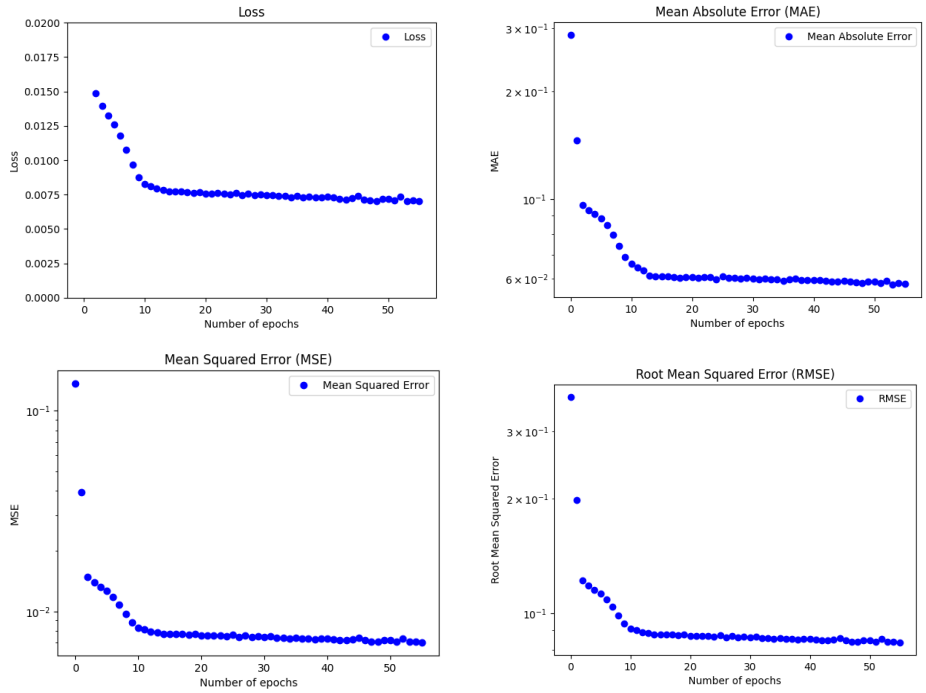

Figure 10: Loss, MAE, MSE, and RMSE measures after data splitting. All four scores drastically decreased during the first 10 epochs, then negligibly decreased before EarlyStopping stopped running the model after 58 epochs.

5 Applying Augmentation to the Model

To augment the data, three different methods were separately applied to the model: time shifting[6], noise injection[6], and time warping[6]. *Time shifting* is the process of shifting the stock data with respect to a predetermined number of days. *Noise injection* is the inclusion of random noise to the data, and *time warping* involves the dilation or compression of time data (i.e., dates).

Negligible differences were found in the performance of the augmented models in comparison to the original model. While all three augmentation methods were ineffective, *noise injection* was the only technique that seemed to benefit the model. The final metric evaluations were as follows, the validation loss was 0.0067, the validation MAE was 0.0643, the validation MSE was 0.0067, and the validation RMSE was 0.0817. The model with *noise injection* will be examined to see if regularization can further improve its performance.

	Loss	MAE	MSE	RMSE
Original	0.0076	0.0603	0.0076	0.0890
Time Shifting	0.0081	0.0654	0.0081	0.0899
Noise Injection	0.0067	0.0643	0.0067	0.0817
Time Warping	0.0080	0.0668	0.0080	0.0896

Figure 11: Tabular metric comparisons of the models with data augmentation to the original model from Section 4 of the report.

6 Applying Regularization to the Model

Through the use of regularization methods such as batch normalization, L1/L2 regularization, and dropout, the model responded the most favorably to the batch normalization. This observation is especially true when adding 3 batch normalization layers to the neural network. After the model finished executing, the validation loss became 0.0041, the validation MAE became 0.0508, the validation MSE became 0.0041, and the validation RMSE became 0.0642.

BatchNormalization

Number of Layers	Loss	MAE	MSE	RMSE
0	0.0067	0.0643	0.0067	0.0817
1	0.0057	0.0596	0.0057	0.0753
2	0.0052	0.0500	0.0052	0.0720
3	0.0041	0.0508	0.0041	0.0642

Figure 12: Validation results after applying different batch normalization layer quantities to the model. The second row of the table (Number of Layers: 0) represents the model *before* batch normalization was applied.

L1 and L2 Regularization

Quantities	Loss	MAE	MSE	RMSE
L1=0.000, L2=0.000	0.0067	0.0643	0.0067	0.0817
L1=0.001, L2=0.001	0.0117	0.0693	0.0087	0.0933
L1=0.002, L2=0.002	0.0136	0.0685	0.0081	0.0898
L1=0.005, L2=0.005	0.0145	0.0712	0.0088	0.0938
L1=0.01, L2=0.01	0.0185	0.0724	0.0087	0.0933
L1=0.05, L2=0.05	0.0991	0.1750	0.0691	0.2629

Figure 13: Validation results after applying different L1 and L2 regularization amounts to the model. The second row of the table (Quantities: L1=0.000, L2=0.000) represents the model *before* L1 or L2 regularization was applied.

Dropout

Quantity	Loss	MAE	MSE	RMSE
0	0.0067	0.0643	0.0067	0.0817
0.01	0.0067	0.0624	0.0624	0.0821
0.05	0.0091	0.0692	0.0091	0.0954
0.1	0.0082	0.0653	0.0082	0.0906
0.2	0.0096	0.0702	0.0096	0.0982
0.5	0.0208	0.0994	0.0208	0.1444

Figure 14: Validation results after applying different dropout amounts to the model. The second row of the table (Quantity: 0) represents the model *before* dropout was applied.

Upon further inspection of the model with 3 batch normalization layers, it is apparent that the Bitcoin price predictions were extremely variable between each day. This scenario is illustrated by the increasingly nonlinear graph patterns in Figure 15 as more batch normalization layers are added to the model. In Figure 16, the price predictions' standard deviations drastically increase with the addition of more layers. It is counter-intuitive to consider this neural network as ideal. As a matter of fact, using *any* batch normalization layers results in blatantly variable and inaccurate results as shown in Figure 15. Therefore, we reject the use of this regularization method.

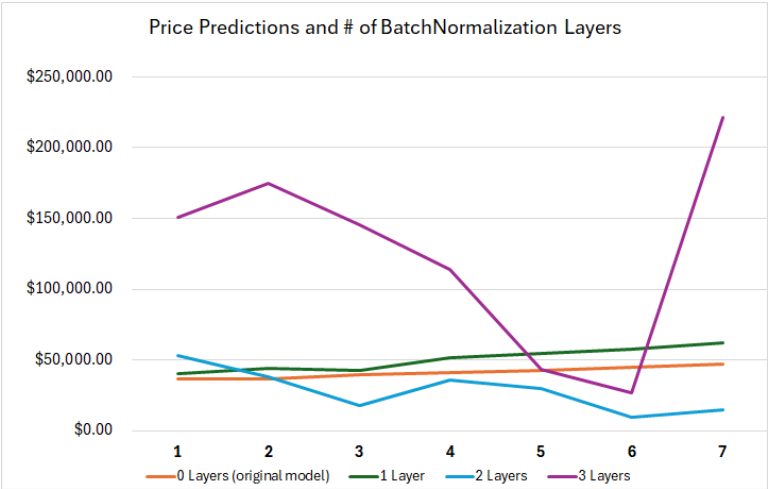

Figure 15: BTC price predictions generated by four models, each with a unique number of batch normalization layers.

Day	0 Layers (original model)	1 Layer	2 Layer	3 Layers
Price Difference	$ 10,329.87	$ 21,365.41	- $ 38,424.03	$ 70,392.99
Percent Change	28.15%	52.93%	-72.69%	46.58%
Mean	$ 41,178.17	$ 50,228.17	$28,230.98	$ 125,343.99
Standard Deviation	3580.02	7653.64	14253.85	64770.40

Figure 16: Statistics of the prices from Figure 15.

After analyzing the predictions' graphical patterns in Figure 15, it turns out that the model with a 0.01 dropout rate offers the best performance and predictions. The final metrics of the neural network are as follows: a validation loss of 0.0067, a validation MAE of 0.0643, a validation MSE of 0.0067, and a validation RMSE of 0.0817. Although these metric scores are higher (worse) than the scores for models with batch normalization, the Bitcoin price predictions are more reasonably patterned, if inaccurate. Figure 17 compares the Bitcoin price predictions for models with a dropout rate of 0.01 and without dropout.

Day	Date	Dropout = 0 (original model)	Dropout = 0.01
1	5/4/2024	$36,698.00	$ 42,452.64
2	5/5/2024	$36,762.44	$ 43,028.07
3	5/6/2024	$39,706.94	$ 44,010.86
4	5/7/2024	$41,061.03	$ 44,943.80
5	5/8/2024	$42,268.93	$ 47,322.19
6	5/9/2024	$44,721.96	$ 49,997.40
7	5/10/2024	$47,027.87	$ 52,483.30
Price Difference		$10,329.87	$10,030.66
Percentage		+28.15%	23.63%
Average		$37,322.13	$46,319.75
Standard Deviation		3580.02	3495.57

Figure 17: Comparison of day-by-day BTC price predictions from models with and without dropout.

7 Applying Different Types of Model Architectures

In previous sections of this report, an LSTM-powered model with MinMaxScaler normalization was trained on Bitcoin time-series data. Different augmentation and regularization techniques were performed–however, the same type of model was used throughout the project. This section deals with different models–ones that are not necessarily LSTM-powered. The following time-series based model layers were tested: Conv1D, SimpleRNN, GRU model, and Bidirectional LSTM.

The Conv1D model performed quite poorly compared to the other models due to its significantly higher performance metric values (Figure 18). This type of model was not designed for time-series, so there was no reason for it to predict Bitcoin prices effectively. The SimpleRNN and Bidirectional LSTM models both obtained similar metric values; the LSTM model that was examined and improved throughout this project also performed similarly to both the other models. The SimpleRNN model turned out to perform the most effectively, as it received a validation loss of 0.0061, a validation MAE of 0.0589, a validation MSE of 0.0061, and a validation RMSE of 0.0764.

Other Model Architectures:

	Loss	MAE	MSE	RMSE
Best Model from Step 6	0.0067	0.0624	0.0624	0.0821
Conv1D	0.0208	0.0994	0.0208	0.1444
SimpleRNN	0.0061	0.0589	0.0061	0.0764
GRU	0.0081	0.0649	0.0081	0.0900
Bidirectional LSTM	0.0062	0.0588	0.0062	0.0788

Figure 18: Validation results after testing different types of neural networks.

For the SimpleRNN model, the four metrics (Figure 19) experienced a sharp decline during the initial 4 epochs, meaning that the model starts learning quickly. The neural network's performance did not seem to improve much after the fourth epoch, so the EarlyStopping callback was activated after epoch 21.

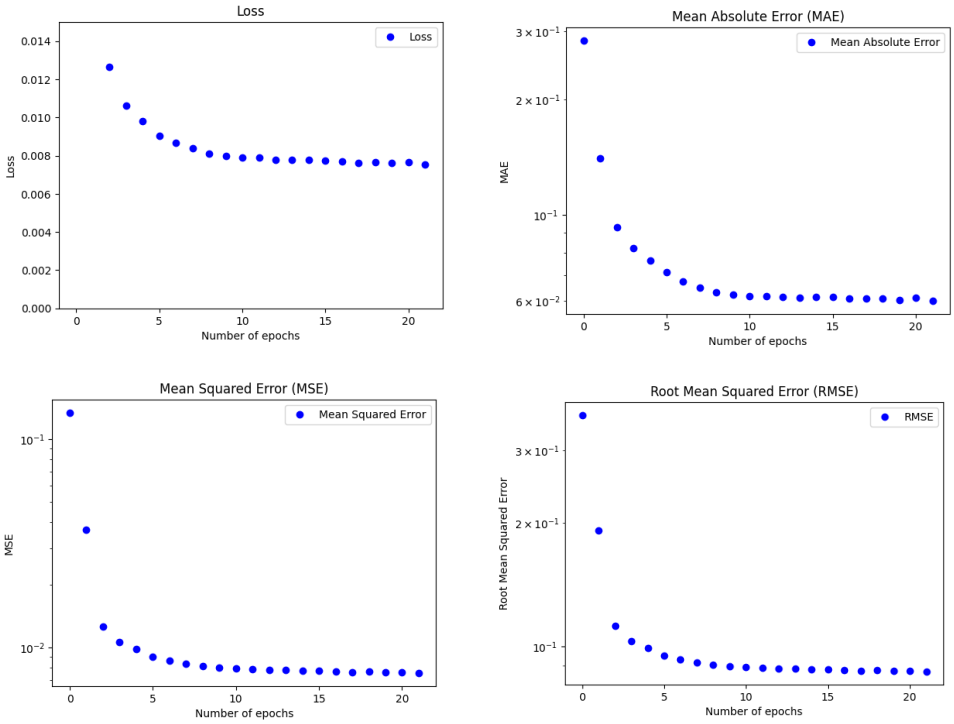

Figure 19: Loss, MAE, MSE, and RMSE measures once the SimpleRNN model completed 21 epochs.

Day	Date	Conv1D	SimpleRNN	GRU	Bidirectional RNN
1	5/4/2024	$32,105.51	$52,728.85	$45,562.29	$40,787.75
2	5/5/2024	$29,559.93	$52,868.01	$45,667.48	$44,420.18
3	5/6/2024	$29,453.70	$53,773.52	$45,412.37	$44,794.46
4	5/7/2024	$33,114.93	$53,828.31	$46,326.75	$48,953.51
5	5/8/2024	$31,715.93	$57,143.35	$47,862.18	$53,270.82
6	5/9/2024	$31,120.73	$57,680.73	$47,822.50	$58,694.11
7	5/10/2024	$28,701.22	$57,918.30	$48,671.79	$65,766.08
Price Difference		- $3,404.29	$5,189.45	$3,109.50	$24,978.33
Percentage		+10.60%	+9.84%	+6.82	+61.24
Mean		$30,824.56	$55,134.44	$46,760.77	$50,955.27
Standard Deviation		1500.29	2163.03	1232.34	8217.23

Figure 20: Comparison of day-by-day BTC price predictions from four different models.

Figure 20 shows that the Price Difference (i.e., the price difference between 5/4/2024 and 5/10/2024), Percentage, Mean, and Standard Deviation are the lowest for the GRU model. This observation signifies that the GRU model produces the least drastic price predictions. One would believe that the GRU model would be the most optimal one because of that observation. However, Bitcoin's current patternicity is not extremely volatile like the model suggests. It seems unreasonable that Bitcoin prices would drop from around $62,000 to around $45,000 (around 27 percent) in one day (from 5/2 to 5/3). Therefore, we reject claiming the GRU model as the best one.

The GRU, Conv1D, and Bidirectional LSTM all predict extremely drastic price plunges between 5/2/2024 and 5/3/2024, as illustrated in Figure 21. However, Bitcoin's actual closing price was $62,889.84 in 5/3/2024. The SimpleRNN model predicted a value of $52,728.05 for that same day, so that model's prediction was closest to the actual price. Therefore, it can be said that the SimpleRNN model was the most optimal throughout this experiment.

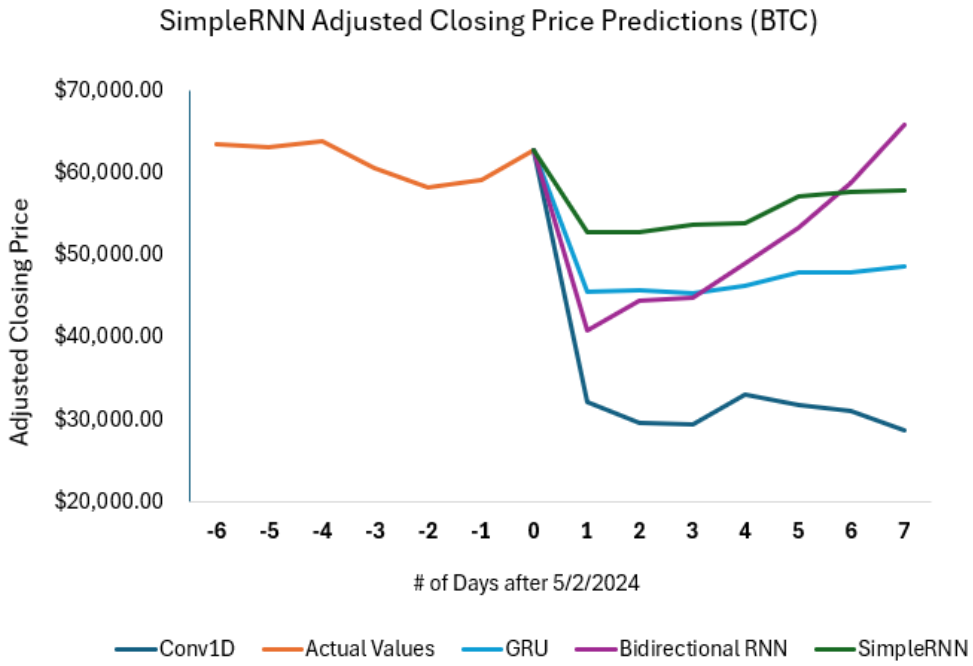

Figure 21: Graphical representation of day-by-day BTC price predictions from the four different models. These predictions are connected to the actual BTC prices that occurred one week before 5/2/2024.

8 Conclusion

When working with Bitcoin time-series data, the final metrics acquired by each model illustrate that they were incapable of accurately predicting cryptocurrency prices. The loss, MAE, MSE, and RMSE scores for each neural network were hardly able to beat their common-sense baselines, if at all. The baselines were 0.06 for the Mean Absolute Error (MAE), 0.01 for the Mean Squared Error (MSE), and 0.09 for the Root Mean Squared Error (RMSE). The SimpleRNN model with noise injection augmentation and a 0.01 dropout rate proved to be the best model throughout this experiment. However, the model only managed to acquire an MAE of 0.0589, an MSE of 0.0061, and an RMSE of 0.0764. These metrics may beat the common-sense baseline, but obtaining metrics that are extremely far from the baseline is the intent of designing and improving models.

Compared to the actual closing prices of $62,889.94 on 5/3/2024, $63,891.47 on 5/4/2024, and $64,031.13 on 5/5/2024, every prediction appeared to present drastic underestimates. The SimpleRNN neural network managed to predict values closest to the actual closing costs, as illustrated in Figures 20 and 21. However, calculating the difference between actual and predicted values presents price differences over $10,000. Price differences were much larger for other types of neural networks. These observations prove that predicting cryptocurrency prices is an extremely difficult Deep Learning problem.

It is far simpler to use time-series models in Deep Learning to forecast traditional stock or ETF prices. Staying informed on a business's decisions and practices can help one make a wise stock investment. Internal factors within an organization obviously influence the rise and fall of stock prices. The risks toward trading stocks are also generally much lower, plus there is much less volatility involved. For these reasons, price patternicity may not be a major issue when applying Deep Learning to stock prediction problems. That is not the case with cryptocurrency. Perhaps it is best to find some alternative avenue toward solving the crypto forecasting problem.

References

[1] Oxford University Press. (n.d.). Cryptocurrency. In *Oxford Learner's Dictionary*. Retrieved May 4, 2024, from https://www.oxfordlearnersdictionaries.com/us/definition/english/food?q=food

[2] Kharif, O. (2017, December 1). *Understanding Bitcoin's rise, from $0.01 to $11,000*. Bloomberg. Retrieved from https://www.bloomberg.com/news/articles/2017-12-01/understanding-bitcoin-s-rise-0-01-to-11-000-quicktake-q-a?embedded-checkout=true

[3] Yahoo! (2024, May 3). *Bitcoin USD (BTC-USD)*. Yahoo! Finance. https://finance.yahoo.com/quote/BTC-USD/history/? guccounter=1period1=1556841600period2=1714767878

[4] GeeksforGeeks. (n.d.). *Data pre-processing with Sklearn using Standard and Minmax Scaler*. Retrieved from https://www.geeksforgeeks.org/data-pre-processing-wit-sklearn-using-standard-and-minmax-scaler/

[5] Francois Chollet. *Deep Learning with Python*. Manning Publications, 2021.

[6] OpenAI. (2023). *ChatGPT* (Feb 13 version) [Large language model]. https://chatgpt.com/c/6d0209df-e148-48e4-87e6-ac9ab8eae46c